Introdução

Algoritmos e Estruturas de Dados

Prof. Dr. Carlos Melo

carlos.melo@upe.br

UPE UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS

Por que estudar algoritmos?

Todo sistema computacional funciona com **DADOS + ALGORITMOS**:

Ordenação:
Organização logística e inventário.

Buscas: Indexação e
recuperação de informação.

Aprendizado de Máquina: Analisa histórico
(Entrada) para predizer preferências (Saída).

Grafos: Calcular
o menor caminho em tempo real.

carlos.melo@upe.br

Bora resolver um problema

Você recebe um número **N**...

- *Como descobrir se ele é* **PAR** *ou* **ÍMPAR**?

```
1.  número ← N
2.  Calcula número % 2
3.  Se resto = 0 → PAR
4.  Senão → ÍMPAR
```

Isso nada mais é que um algoritmo

O que é um algoritmo?

Um **algoritmo** é qualquer procedimento computacional ***bem definido*** que toma algum valor ou conjunto de valores como **entrada** e produz algum valor como **saída.**

Ideia principal: Entrada -> Passos bem definidos -> Saída!

carlos.melo@upe.br

Isso também é um algoritmo?

- Aqueça o forno à 180°C;
- Unte uma forma redonda;
- Numa taça bata até ficar cremoso:
  - 75g de manteiga;
  - 250g de açúcar.
- Junte:
  - 4 ovos, um a um;
  - 100g de chocolate derretido;
  - Adicione aos poucos 250g de farinha peneirada.
- Deite a massa na forma;
- Leve ao forno durante 40 minutos.

Objetivos da Disciplina

Objetivos

Nesta disciplina vocês aprenderão à:

- Resolver problemas usando algoritmos
- Organizar dados de forma eficiente
- Comparar soluções e analisar eficiência
- Implementar algoritmos e estruturas de dados

Sobre a disciplina

Metodologia e Avaliação

- Aprendizagem ativa
  - vocês serão os protagonistas no próprio aprendizado
- Baseada em Projetos
  - mão na massa
- Avaliação contínua por meio de:
  - Presença e participação (Pode ajudar no arredondamento);
  - Listas de exercício (10%);
  - Projeto (30%);
  - Provas (60%).

Sobre a disciplina

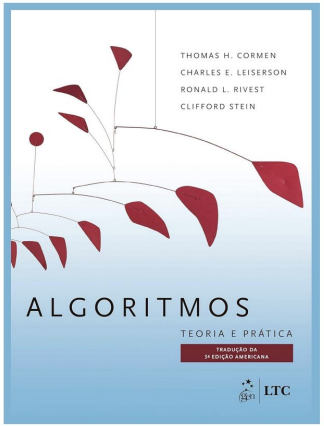
Presencial

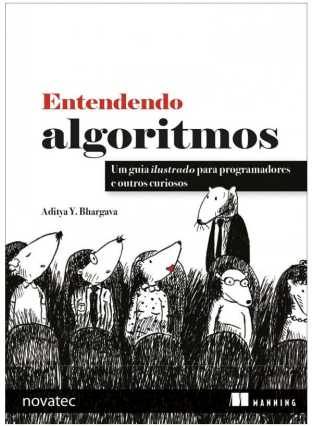
- 4 aulas por semana
  - *sempre às quartas-feiras*
- Teremos chamada
  - *com* **presença obrigatória**
- Teremos Monitoria (*terças e quintas das 19 às 20:30*)

Sobre a disciplina

Bibliografia Básica

- CORMEN, T. H., LEISERSON, C. E., RIVEST, R. L., & STEIN, C. (2002). **Algoritmos: teoria e prática**. Editora Campus, 3a Edição, 2012.
- BHARGAVA, Aditya Y. **Entendendo Algoritmos** : Um guia ilustrado para programadores e outros curiosos. Novatec Editora, 2018.

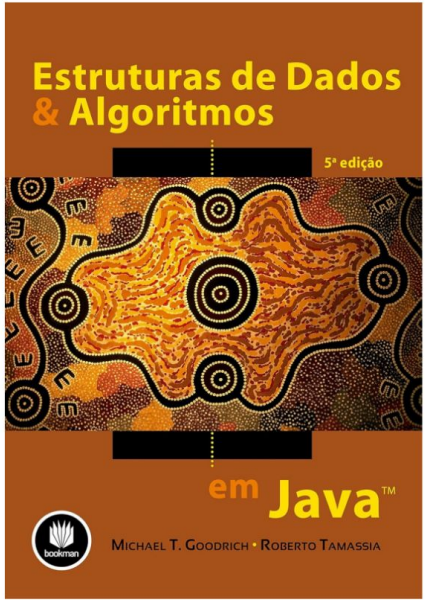


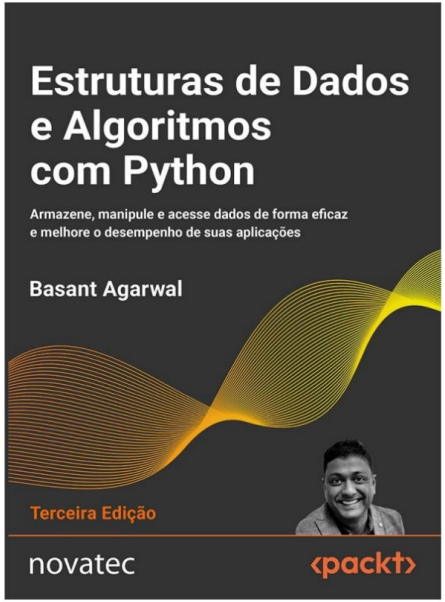


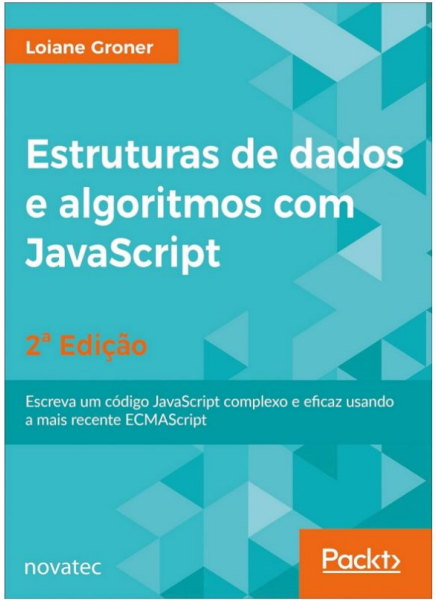
carlos.melo@upe.br

Sobre a disciplina

Bibliografia Complementar







carlos.melo@upe.br

Conteúdo do Curso

1. Introdução, Pesquisa Binária e Big O
2. Arrays vs. Listas Encadeadas
3. Pilhas, Filas e Recursão
4. Divisão e Conquista
5. Tabelas Hash
6. Árvores, Heaps e Heapsort
7. Algoritmos de Ordenação Linear
8. Grafos e Busca em Largura (BFS)
9. Grafos Ponderados - Dijkstra
10. Busca em Profundidade (DFS) e Aplicações
11. Árvore Geradora Mínima (Prim/Kruskal)
12. Algoritmos Gulosos e Programação Dinâmica
13. Casamento de Padrões e Strings (*rolling hash*)
14. NP-Completude

Algoritmo

Além disso, os algoritmos se baseiam no uso de um conjunto de instruções ***bem definido***, que constituem um **vocabulário** de símbolos limitado (por exemplo, o conjunto de palavras reservadas da linguagem).

```python
def par_ou_impar(n):
    if n % 2 == 0:
        print("par")
    else:
        print("impar")
```

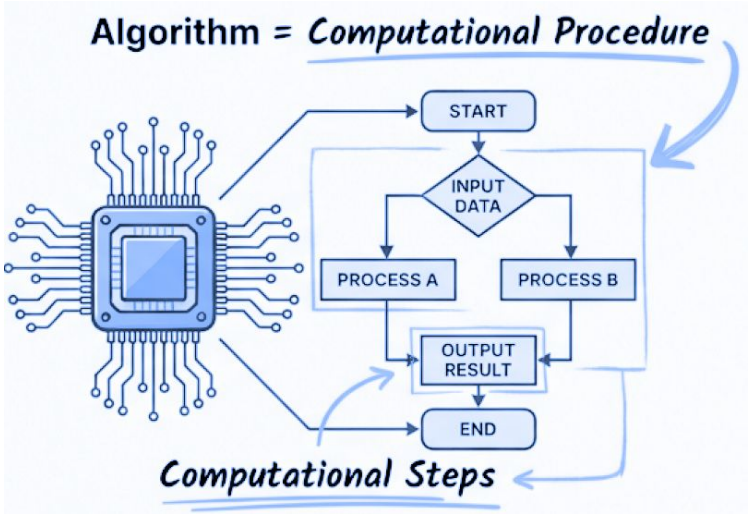
Exemplos comuns: if, else, continue, break, while, for, def, return...

Algoritmo

A sequência de instruções que define o algoritmo deve ser sempre **finita** e também não pode ser ambígua.

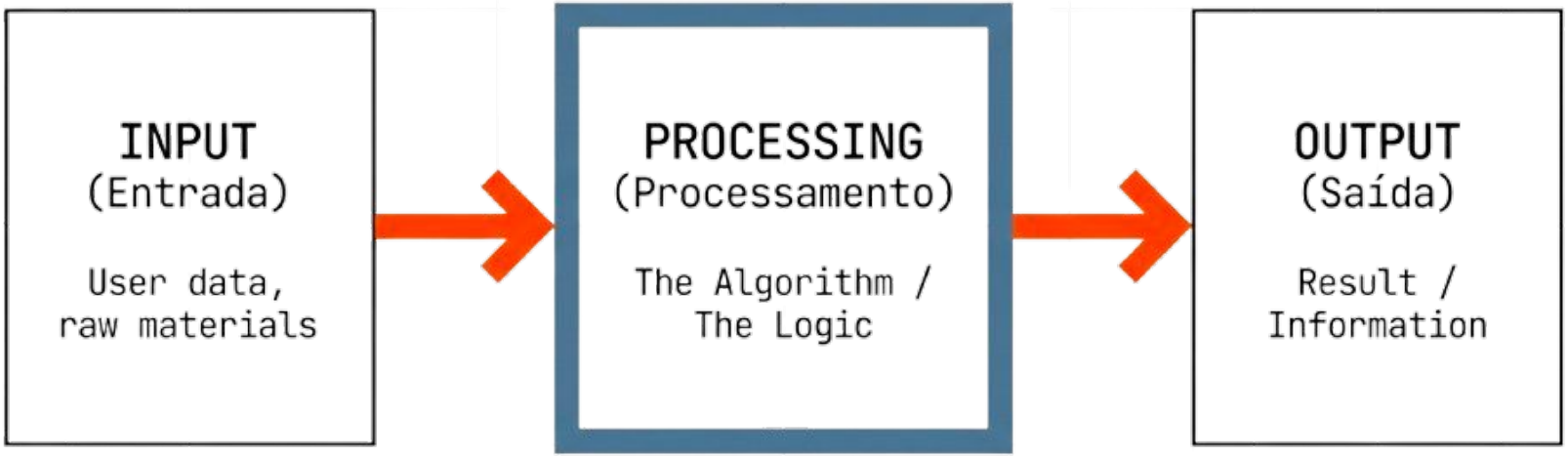
Isso significa que **nosso algoritmo deve ter**:

- **sempre um fim** (deve terminar após determinado número de passos) e que;
- cada instrução deve ser **precisamente definida** (não deve permitir mais de uma interpretação de seu significado).



Algoritmo

- Algoritmos são compostos de 3 partes:
  - **Entrada de dados** : são os dados do algoritmo informados pelo **usuário**;
  - **Processamento de dados** : são os procedimentos utilizados para chegar ao resultado;
  - **Saída de dados** : são os dados já processados, apresentados pelo usuário.



Neste curso vamos focar na acurácia e na eficiência do bloco de Processamento

carlos.melo@upe.br

Corretude de Algoritmos

Muitas vezes, um ***mesmo problema*** pode ser resolvido por **vários algoritmos** diferentes...

Mas... nem sempre a forma que escolhemos nos retornará o resultado esperado:

- não é porque minha tarefa é remover um parafuso que sempre devo utilizar uma chave de fenda para tal.
- E muito menos porque um ícone dos quadrinhos veste a cueca por cima da calça que eu devo fazê-lo.

Ou seja, um algoritmo pode ou não ser **correto**.

carlos.melo@upe.br

Corretude de Algoritmos

Um algoritmo é considerado correto se, e somente se, nos entrega a saída esperada para **todas** as entradas (*instâncias*) daquele problema, ou seja, ele resolve aquele problema.

Porém, *às vezes*, não há uma solução correta para determinado problema e devemos nos contentar com algoritmos ***incorretos*** desde que a taxa de erro seja controlável porque podem fornecer uma solução aproximada uma vez que a solução exata possa ser muito difícil de encontrar.

-> Acurácia

Eficiência de Algoritmos

Um algoritmo pode ser correto, mas ainda assim ser muito lento, ou seja, é **ineficiente**.

-> **Eficiência significa resolver o problema usando menos tempo ou menos memória.**

-> Refere-se a "fazer as coisas da maneira certa", otimizando processos.

-> Foca na relação entre insumos (recursos) e produtos (resultados).

Eficiência de Algoritmos

Os algoritmos se diferenciam pela forma como utilizam recursos do computador.

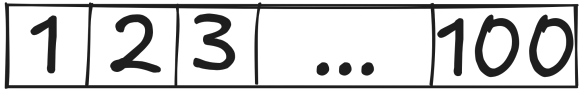
Existem **algoritmos que dependem do tempo** de execução, enquanto outros dependem **da quantidade de memória**.

-> um bom algoritmo busca sempre a otimização de **recursos**, uma vez que estes também são **finitos**.

Exemplos de problemas comuns e com as mais variadas eficiências com vários algoritmos corretos para a sua resolução são os de **busca e ordenação** .

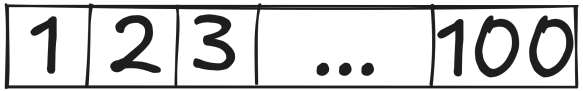
Problemas (Buscando Valores Específicos)

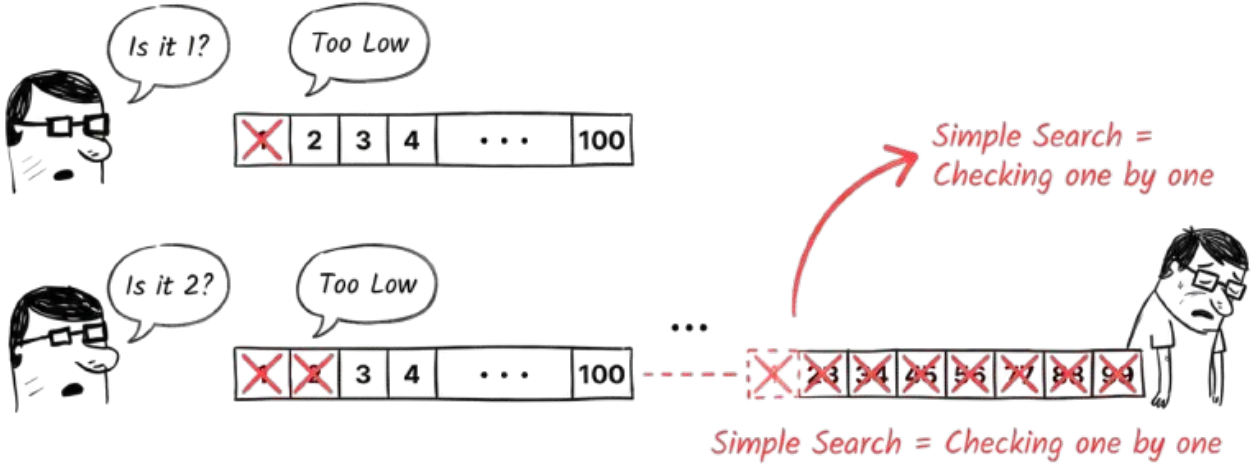
Adivinhe o número:



A busca simples

Adivinhe o número:

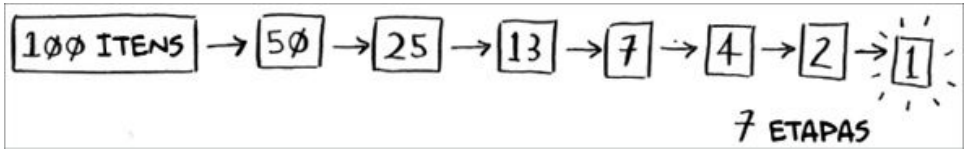




No pior caso: se o número é 100, você precisa de 100 chutes para acertar. Para *n* números, você precisa de *n* passos

A busca binária

Elimine metade dos números a cada tentativa com a busca binária



Eficiência: Para 100 itens, nunca precisaremos de mais do que 7 chutes. (2^7 = 128)

carlos.melo@upe.br

Exercício

1. Você precisa realizar a busca por um convidado em uma lista com 128 nomes. Qual seria o número máximo de passos até encontrar o nome desejado usando a busca simples? E a busca binária?

2. Agora, imagine que cada convidado levará seu cônjuge consigo. Qual seria o número máximo de passos agora?

1. **Busca simples**: 128 passos. **Busca binária**: 7 passos (2^7 = 128).

2. **Busca simples**: 256 passos. **Busca binária**: 8 passos (2^8 = 256).

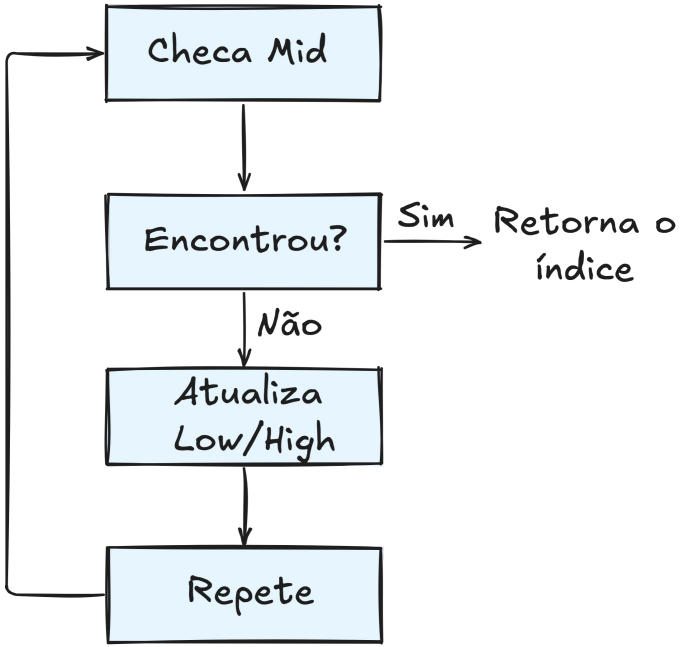
E se fossem 1 bilhão de convidados?

Busca simples: 1 bilhão de passos. **Busca binária**: 30 passos (2^30 = 1.073.741.824).

Qual o algoritmo da busca binária?

```python
def binary_search(list, item):
    low = 0
    high = len(list) - 1
    while low <= high:
        mid = (low + high) // 2
        guess = list[mid]
        if guess == item:
            return mid
        if guess > item:
            high = mid - 1
        else:
            low = mid + 1
    return None
```

Integer Division (rounds down)



Logaritmos

Logaritmos são o ***oposto*** dos Exponenciais.

- A expressão $\log_{10}100$ nos diz: "Quantos 10s conseguimos multiplicar para chegar a 100?".

Porque 10x10 = 100 então $\log_{10}100 = 2$.

Uma outra forma de vermos isso é: até quantas vezes posso dividir o alvo pelo número base?

- $\log_2 n$: Quantas vezes podemos dividir **n** por **2** até chegar a 1?
- Se **n = 128** então: **128 -> (64 -> 32 -> 16 -> 8 -> 4 -> 2 -> 1)** (7 vezes), ou seja $\log_2 128 = 7$

Exercício

1. $\log_{10} 1000$?
2. $\log_2 8$?
3. $\log_2 16$?
4. $\log_2 32$?

1. 100 -> 10 -> 1 (3 vezes)
2. 4 -> 2 -> 1 (3 vezes)
3. 8 -> 4 -> 2 -> 1 (4 vezes)
4. 16 -> 8 -> 4 -> 2 -> 1 (5 vezes)

A busca binária

De maneira geral, para uma lista de n números, a busca BINÁRIA precisa de $\log_2 n$ passos para retornar o valor correto, enquanto que a busca simples precisa de n passos.

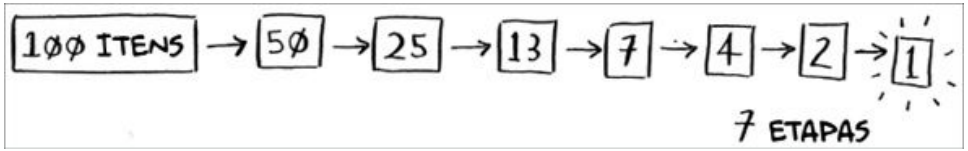
Porém... seria a busca binária a bala de prata das buscas?

Na verdade a busca binária só funciona para entradas ordenadas...

Aplicações de algoritmos

Busca binária:

-> **Aplicação:** Busca eficiente em bancos de dados ordenados, localização de itens em listas ou arrays.
-> **Exemplo Real:** O Google usa busca binária em seus índices para fornecer resultados de pesquisa rápidos e eficientes, localizando informações relevantes em grandes conjuntos de dados.



Aplicações de algoritmos

Ordenação de dados:

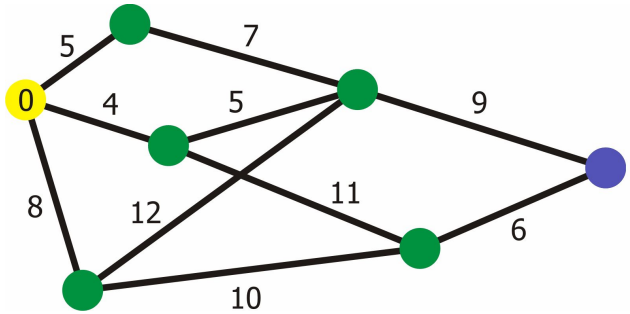
-> **Aplicação:** Classificação de resultados de pesquisa, organização de dados em bancos de dados, ordenação de elementos em listas.
-> **Exemplo Real:** A Amazon utiliza algoritmos de ordenação para classificar resultados de pesquisa, apresentar produtos em ordem relevante para o usuário e otimizar a eficiência em seus armazéns.

Selection Sort

Aplicações de algoritmos

Algoritmos de caminho mínimo:

-> **Aplicação:** Roteamento em redes de computadores, navegação em mapas, otimização de rotas para entrega de mercadorias.
-> **Exemplo Real:** Empresas de transporte/entregas, como a Uber, utilizam algoritmos de caminho mínimo para otimizar as rotas de motoristas, garantindo uma entrega eficiente em termos de tempo e custo.

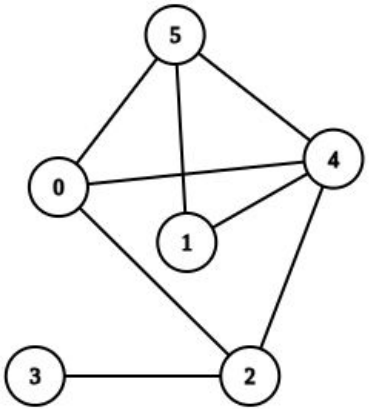


Algoritmo de Dijkstra

Aplicações de algoritmos

Algoritmos de grafos:

-> **Aplicação:** Redes sociais, análise de redes, roteamento em sistemas de transporte público.
-> **Exemplo Real:** O Facebook utiliza algoritmos de grafos para sugerir conexões entre usuários com base em amizades mútuas, analisando a estrutura de redes sociais.

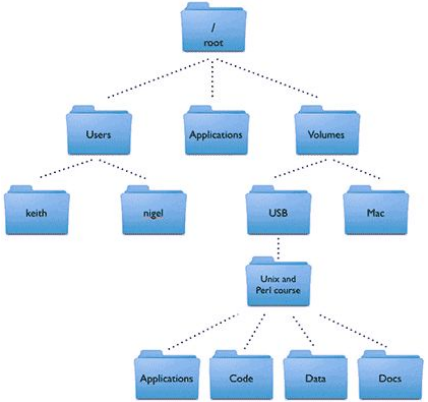


carlos.melo@upe.br

Aplicações de algoritmos

Algoritmos de árvores:

-> **Aplicação:** Estruturas de diretórios de sistemas de arquivos, otimização de operações em bancos de dados, representação de hierarquias em organizações.
-> **Exemplo Real:** Bancos como o Wells Fargo podem usar árvores para organizar a estrutura de diretórios em sistemas de arquivos facilitando o acesso rápido e eficiente aos dados.

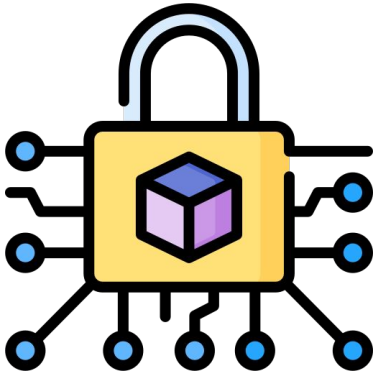


carlos.melo@upe.br

Aplicações de algoritmos

Algoritmos de criptografia:

-> **Aplicação:** Segurança em comunicações online, autenticação, proteção de dados sensíveis.
-> **Exemplo Real:** O WhatsApp aplica algoritmos criptográficos para garantir a privacidade das mensagens, protegendo a comunicação online de usuários.

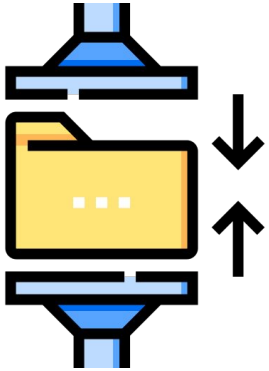


carlos.melo@upe.br

Aplicações de algoritmos

Algoritmos de compressão de dados:

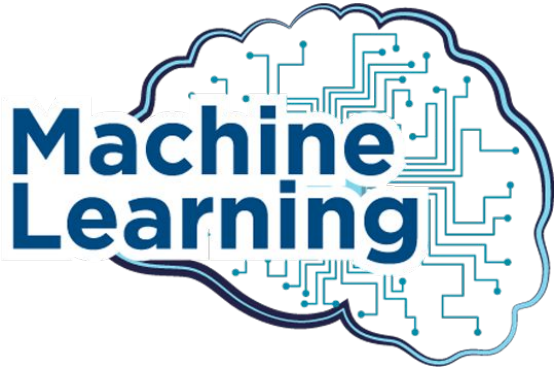
-> **Aplicação:** Compactação de arquivos para economizar espaço de armazenamento, transmissão eficiente de de dados pela internet.
-> **Exemplo Real:** Plataformas de streaming, como o Netflix, utilizam algoritmos de compressão para transmitir vídeos de alta qualidade de maneira eficiente pela internet, economizando largura de banda.



Aplicações de algoritmos

Algoritmos de aprendizado de máquina:

-> **Aplicação:** Classificação de emails como spam ou não spam, recomendações de produtos em plataformas de comércio eletrônico, reconhecimento de padrões em imagens.
-> **Exemplo Real:** A Netflix utiliza algoritmos de aprendizado de máquina para recomendar filmes e séries com base no histórico de visualização do usuário, melhorando a experiência de usuário.

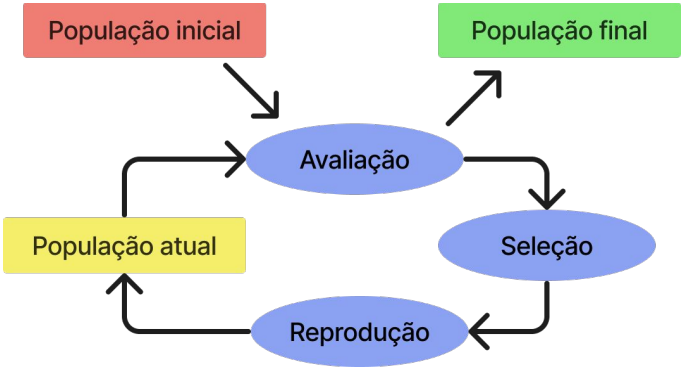


carlos.melo@upe.br

Aplicações de algoritmos

Algoritmos genéticos:

-> **Aplicação:** Otimização de processos, design de sistemas, resolução de problemas complexos.
-> **Exemplo Real:** Empresas de design automotivo como a Tesla, podem usar algoritmos genéticos para otimizar o design de carros, levando em consideração diversos parâmetros de desempenho.



Aplicações de algoritmos

Algoritmos processamento de imagens:

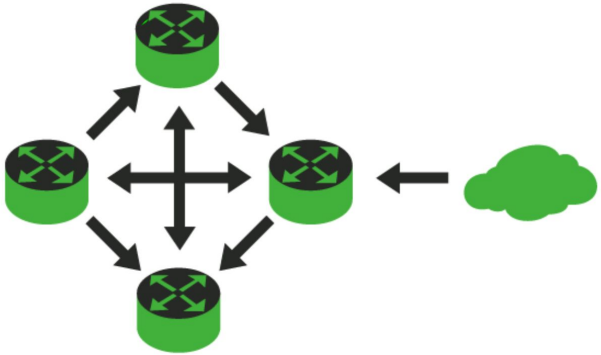
-> **Aplicação:** Reconhecimento facial, melhoria de qualidade de imagem, segmentação de objetos.
-> **Exemplo Real:** O Instagram aplica algoritmos de processamento de imagens para reconhecimento facial, aplicação de filtros e melhoria automática da qualidade visual das fotos.

carlos.melo@upe.br

Aplicações de algoritmos

Algoritmos de roteamento em redes:

-> **Aplicação:** Roteamento de pacotes de dados na internet, gerenciamento de tráfego em redes de comunicação.
-> **Exemplo Real:** Empresas de telecomunicações, como a AT&T, utilizam algoritmos de roteamento para gerenciar o tráfego de dados em suas redes, garantindo uma comunicação eficiente.



carlos.melo@upe.br

Aplicações de algoritmos

Algoritmos de otimização:

-> **Aplicação:** Otimização de rotas de entrega, planejamento de produção, design de redes de transporte.
-> **Exemplo Real:** Empresas de logística, como a UPS, empregam algoritmos de otimização para planejar rotas de entrega, minimizando o tempo e os custos envolvidos no transporte de mercadorias.

Aplicações de algoritmos

Algoritmos de diagnóstico médico:

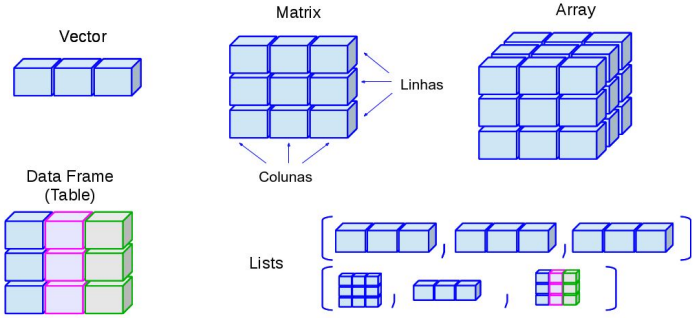
-> **Aplicação:** diagnóstico de doenças como câncer e tuberculose.
-> **Exemplo Real:** Hospitais e Laboratórios de análises clínicas de todo o Brasil como Hermes Pardini, Dasa e o grupo Fleury.

Estruturas de Dados

Uma **estrutura de dados**, em computação, é uma forma de armazenar e organizar os dados de modo que eles possam ser utilizados de forma **eficiente**.

-> Uma estrutura de dados é um relacionamento lógico entre diferentes tipos de dados visando a resolução de determinado problema de forma eficiente.

-> Algoritmos processam dados. Estruturas de dados organizam esses dados.



Estruturas de Dados

Uma **estrutura de dados** é uma forma eficiente de armazenar e organizar os dados dentro do computador para que nossos algoritmos tirem o melhor proveito deles.

-> a organização de forma coerente dos dados permite a redução do custo de execução de um algoritmo em termos de **tempo de execução, consumo de memória** ou ambos, ou seja, o correto uso de estruturas de dados pode aumentar o desempenho de um algoritmo.

-> Finada de contatos:
  -> ordenada (busca rápida)
  -> desordenada (busca lenta)

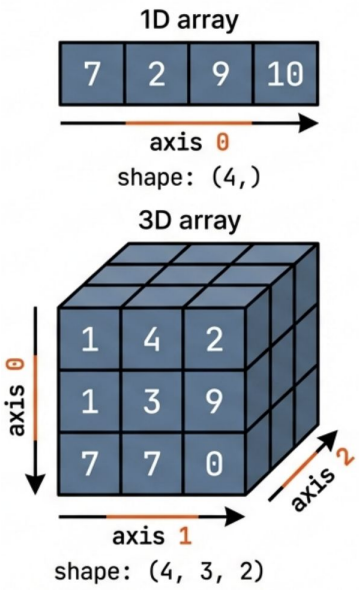
Estruturas de Dados

Diferentes tipos de estruturas são adequados a diversos tipos de aplicações, sendo algumas altamente especializadas para tarefas específicas.

Exemplos de estruturas de dados:

-> Arrays;
-> Listas encadeadas (ou ligadas);
-> Filas e Pilhas;
-> Árvores;
-> Tabelas Hash.

Cada tipo de estrutura de dados resolve um tipo de problema.



Estruturas de Dados

Algoritmos importam!

Sugestão de leitura:

-> https://meiobit.com/272167/top7-the-greatest-epic-failures-of-the-computing/

-> https://gizmodo.uol.com.br/hifen-nasa/

-> https://gizmodo.uol.com.br/exomars-falha-de-software/

Erro de software pode ter causado a queda da sonda da ExoMars em Marte

Estudos preliminares indicam que uma falha de software causou a queda da sonda Schiaparelli em Marte na semana passada.

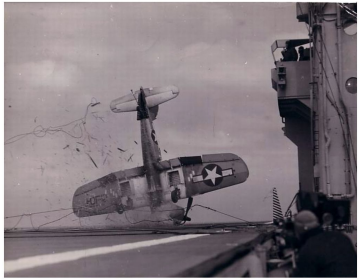
O hífen que destruiu um foguete da NASA

Em 22 de julho de 1962, o Mariner I estava em sua plataforma, pronto para fazer história. Depois do investimento de anos de construção, cálculos e financiamento, a NASA tinha grandes esperanças de que seu foguete realizaria com sucesso um voo de reconhecimento até Vênus, o que daria impulso para a corrida espacial americana. Em [...]

Os 7 maiores Epic Fails da história da computação

Errar é humano, mas pra errar BONITO é preciso um computador. Duvida? Veja então os 7 maiores Epic Fails da informática...

Carlos Cardoso · 12 anos atrás



carlos.melo@upe.br

O que veremos em nosso curso?

Nesta disciplina usaremos: Python 3+

Linguagem Python
[Docs] https://docs.python.org/pt-br/3/

Google Colab
https://colab.google/

Python Tutor
https://pythontutor.com/visualize.html#mode=edit

carlos.melo@upe.br

Prof. Dr. Carlos Melo
https://calendly.com/prof-casm
carlos.melo@upe.br

UPE UNIVERSIDADE DE PERNAMBUCO CAMPUS GARANHUNS